

**VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
SPECIALIZED PROJECT**

SEMESTER 241 ACADEMIC YEAR 2024-2025

**CODERANK SYSTEM
ONLINE PLATFORM FOR ALGORITHM PRACTICE
AND PROGRAMMING CONTEST
FOR VIETNAMESE IT STUDENT**

MAJOR: COMPUTER SCIENCE

THESIS COMMITTEE : SOFTWARE ENGINEERING - 08 CLC

SUPERVISOR(s) : ASSOC. PROF. QUAN THANH THO

STUDENT 1 : NGUYEN CHAU HOANG LONG (2252444)

STUDENT 2 : CAO NGOC LAM (2252419)

HO CHI MINH CITY, DECEMBER 2025



Thank you

I like to acknowledge ...

Contents

1	Introduction	5
1.1	Problem statement	5
1.2	Goals	6
1.3	Scope	7
2	Technology Stack	8
2.1	Sandboxed Execution Environment	9
2.1.1	Firecracker MicroVMs	9
3	System Design and Analysis	10
3.1	Related works	10
3.1.1	Leetcode	10
3.1.2	Codeforces	11
3.1.3	Websites comparison	12
3.2	Requirement analysis	13
3.2.1	Stakeholders	13
3.2.2	Functional requirements	14
3.2.3	Non-functional requirements	16
3.3	Use Cases	17
3.3.1	General Use Case	17
3.3.2	Use-case diagrams for each submodule	18
3.3.3	Use-case description tables	21
3.4	Architecture Diagram	33
3.4.1	Architectural Decision Rationale	33
3.4.2	System Components	35
3.5	Code Runner	36
3.5.1	Overview	36
3.5.2	Class Diagram	37
3.5.3	Testcase Format	38
4	Future Plan and Work Evaluation	40

List of Figures

3.1	Leetcode Homepage	10
3.2	Codeforces Homepage	12
3.3	General Use Case	17
3.4	Authentication & Authorization Use Case	18
3.5	System Management Use Case	19
3.6	Discussion Use Case	20
3.7	Submission Use Case	21
3.8	Architecture Diagram	34
3.9	Layered Architecture	35
3.10	Class Diagram for Code Runner	37

List of Tables

3.1	Comparison of existing website with CodeRank	13
3.2	UC-1: Problems Management	22
3.3	UC-2: Contest Management	23
3.4	Use Case UC-3: Primary User Account Registration	24
3.5	UC-4: Problem Browsing & Searching	25
3.6	Use Case UC-5: Solving a Problem	26
3.7	UC-6: Create a Discussion	27
3.8	UC-7: Participate in a Discussion Thread	28
3.9	UC-8: Edit Profile	29
3.10	UC-9: Participate in a Contest	31
3.11	UC-10: Chatbot Interview Conversation	32
3.12	Data Types for Testcase Fields	38

Chapter 1

Introduction

In Chapter 1, the authors provide a comprehensive introduction to the project's domain, presenting the relevant background and context. Based on this overview, the problem statement is carefully formulated, emphasizing its significance, scope, and the main objectives that the project aims to achieve.

1.1 Problem statement

With the rapid progress in educational technology, the concept of an online coding platform with a ranking system for students is increasingly recognized as a valuable tool to enhance learning outcomes, foster problem-solving skills as well as serving as a solid foundation to prepare students for real-world job interviews in the labor market. While some recognized global platforms such as LeetCode and Codeforces provide extensive resources for competitive programming, there remains a gap in platforms tailored specifically to the needs of Vietnamese students. Current solutions do not fully address the language preferences, localized content, or the collaborative opportunities required to create a holistic learning environment. Consequently, it is essential to enhance the manner in which students approach coding problems, submit their solutions, and collaborate with peers in an effective and intuitive way.

One of the primary challenges lies in problem management and submission tracking. Students require a system that allows them to browse coding problems by category and difficulty, submit solutions with immediate evaluation, and track their performance over time. The lack of a structured and interactive ranking mechanism can reduce motivation and hinder continuous improvement, as students may struggle to assess their progress and compare their performance with peers effectively.

Another significant challenge is fostering real-time collaboration and discussion. Existing

platforms often lack localized forums where students can post questions, discuss problem-solving strategies, and exchange knowledge in their own language. Without an integrated discussion system, students may confront barriers in clarifying ambiguities and engage in collaborative learning, which significantly curtails both overall engagement and the depth of the learning experience.

In summary, the system must be accessible and intuitive, catering to students with various levels of programming expertise. By offering a user-friendly interface, real-time submission feedback, a ranking system, and an integrated discussion forum, the platform can foster an interactive and engaging environment. This approach encourages students to actively participate, receive immediate feedback on their work, and collaborate effectively with their peers, thereby enhancing the overall learning experience. These challenges highlight the necessity of a system designed specifically for Vietnamese students that integrates problem-solving, submission tracking, and collaborative learning in a seamless manner.

1.2 Goals

The main goal of this project is to design and implement a Code Ranking System specifically tailored for Vietnamese students, addressing the challenges identified in the problem statement. The system aims to provide an intuitive and accessible platform that allows undergraduates to effectively engage with coding problems, submit solutions, and interact with their peers. By focusing on usability and real-time feedback, the platform seeks to enhance learning outcomes and foster a collaborative environment.

Firstly, the system will enable students to browse coding problems organized by category and difficulty level, with clear description and examples. Students will be able to submit their solutions and receive immediate evaluation, allowing them to track their progress and monitor improvement over time. A structured ranking mechanism will provide motivation and encourage healthy competition among users.

Secondly, the platform will incorporate real-time centralized discussion features, where students can post questions, share strategies, and collaborate with peers in their native language. This will help clarify ambiguities, promote knowledge sharing, and deepen the learning experience.

Thirdly, the platform will provide a free chatbot feature that allows students to ask questions and engage with the bot regarding topics relevant to coding, problem-solving, and real-world interview scenarios in the Vietnamese job market. This chatbot aims to simulate practical

interview discussions, offer guidance, and help students prepare for professional opportunities.

Finally, the platform will support teachers and moderators with administrative functionalities. These include problem management, monitoring student submissions, and generating performance statistics, thereby providing insights to improve teaching strategies and maintain a structured, high-quality learning environment.

In summary, this project aims to create an interactive, motivating, and collaborative coding platform for Vietnamese students, facilitating skill development, real-time feedback, peer engagement, and practical interview preparation, while bridging the gap between global competitive programming platforms and the local educational context.

1.3 Scope

Our team is developing a web-based platform specifically designed for desktop users, integrating AI-driven features to enhance the learning and coding experience.

- **Website:** By focusing on PC compatibility, the platform ensures smooth navigation and interaction for students. Key modules include problem exploration, solution submission, ranking system, and a centralized discussion forum where students can engage, share insights, and collaborate.
- **AI-powered Assistance:** The system incorporates an intelligent chatbot that helps students with coding guidance and interview preparation. Using natural language processing, the chatbot can interpret questions and provide personalized recommendations, enabling students to improve their skills and gain practical experience in preparing for real-world job opportunities.

Chapter 2

Technology Stack

The purpose of Chapter 2 is to provide a comprehensive technical overview and rationale behind the selection of the platform's diverse technology stack. The authors articulate the specifics of the chosen front-end and back-end frameworks, the database management system, and the integration of the LLM model to facilitate the conversational chatbot feature.

2.1 Sandboxed Execution Environment

In the world of cybersecurity, a **sandbox environment** is an isolated virtual machine in which potentially unsafe software code can execute without affecting network resources or local applications. Think of a sandbox as a controlled playground where applications, code, and files can be tested or executed to see how they behave. If the software behaves maliciously or unexpectedly, it doesn't have the power to affect anything outside of that contained environment. [3]

In the context of CodeRank System, this mechanism is essential because the platform allows users to submit arbitrary source code for compilation and execution as part of algorithm practice and programming contests. Without sandboxing, user submissions could intentionally or accidentally access the file system, overload system resources, execute harmful commands, or interfere with other users' submissions. By running each submission inside a sandbox, CodeRank ensures that all code is contained, resource-restricted, and prevented from performing dangerous operations. This guarantees fair evaluation, protects the platform's infrastructure, and maintains a secure environment for all Vietnamese IT students using the system.

By considering the requirements for securely running user-submitted code, it is essential to evaluate different execution sandbox technologies used in modern online judge systems. In the following section, three representative tools will be introduced and compared in terms of security, performance, isolation level, and ease of integration.

2.1.1 Firecracker MicroVMs

Firecracker is an open source virtualization technology that is purpose-built for creating and managing secure, multi-tenant container and function-based services. Firecracker enables you to deploy workloads in lightweight virtual machines, called microVMs, which provide enhanced security and workload isolation over traditional VMs, while enabling the speed and resource efficiency of containers. Firecracker was developed at Amazon Web Services to improve the customer experience of services like AWS Lambda and AWS Fargate. [2]

Chapter 3

System Design and Analysis

Chapter 3 first provides a comparative analysis of similar applications, evaluating their strengths and limitations. This is followed by a detailed specification of the system requirements and use cases necessary to address the project's core challenges. Finally, the chapter concludes by defining the system's architecture, database schema, AI adaptability, and the finalized wireframe design.

3.1 Related works

3.1.1 Leetcode

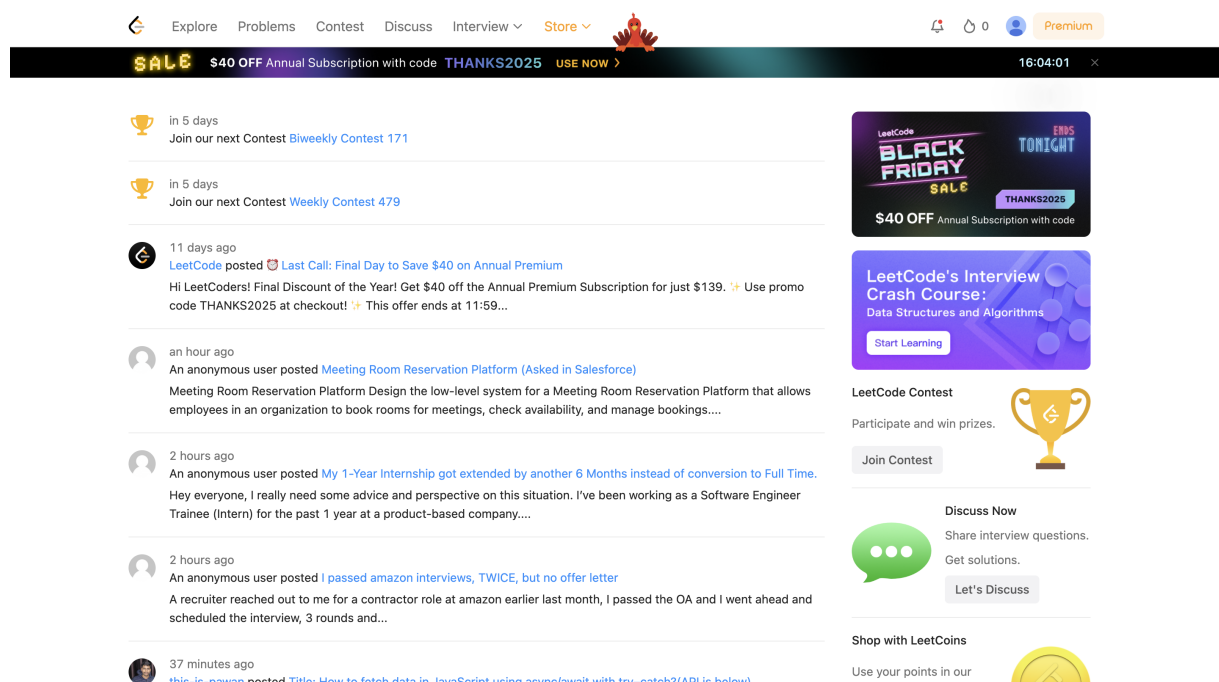


Figure 3.1: Leetcode Homepage

LeetCode, founded in 2015 based on the co-founders' experiences working at Google and Amazon, has become a highly popular online coding platform that offers a large collection of programming problems for all skill levels. It offers a comprehensive environment where users can practice a wide variety of programming problems, engage with a global community, and track their progress over time. LeetCode is widely recognized for its role in fostering problem-solving abilities and strengthening algorithmic thinking, making it an essential tool for both students and professionals in the software engineering field.

Key features of LeetCode include:

- **Extensive Problem Library:** Thousands of coding problems across multiple difficulty levels and topics.
- **Online Code Editor:** Supports multiple programming languages with instant feedback and test case evaluation.
- **Contest and Challenge System:** Regular coding contests to benchmark skills against a global community.
- **Interview Preparation Kits:** Curated problem sets designed to prepare users for technical interviews at top tech companies.
- **Discussion Forums:** Active community discussions for problem-solving strategies, solutions, and learning resources.
- **Progress Tracking and Analytics:** Tools to monitor performance, problem-solving patterns, and skill improvement over time.

LeetCode offers a diverse set of core features that comprehensively address the needs of different user roles, including problem solving, submission tracking, contest participation, community interaction, ranking, and interview preparation. These features collectively enhance the learning experience, promote skill development, and encourage active engagement within the platform. However, it is noteworthy that LeetCode currently does not provide a public leaderboard to foster a stronger competitive environment among users. Additionally, while users can participate in official contests organized by LeetCode, the platform does not support user-generated contests, limiting the flexibility for personalized contest creation.

3.1.2 Codeforces

Codeforces is a prominent online platform dedicated to competitive programming, offering users a structured environment to enhance their algorithmic and problem-solving skills through real-time contests. It is widely used by students, professionals, and competitive programmers to benchmark their abilities and engage with a global community of coders.

Key features of LeetCode include:

- **Extensive Problem Sets:** A vast collection of algorithmic and programming problems

CODEFORCES
Sponsored by TON

longnguyendt04 | Logout

HOME TOP CATALOG CONTESTS GYM PROBLEMSET GROUPS RATING EDU API CALENDAR HELP

OCPC 2026 Winter

By [adamant](#), 5 hours ago,

Hi everyone!

Supported by

The **OCPC** (Osijek Competitive Programming Camp), an [award-winning](#) camp for university students to prepare for ICPC and other coding competitions, is back again this winter, on **January 24 – February 1, 2026!**

The camp is organized by [adamant](#) and [-is-this-ft-](#) on the premises of the University of Osijek in Osijek, Croatia.

The camp is inspired by various competitive programming camps that we attended during our active years in ICPC, and is aimed to help college students prepare for ICPC regional contests and finals. The camp will consist of 7 ICPC-style contests and 2 days off.

Details

The participation fee for both online and onsite participants is **198€ per team**.

Similar to previous camp editions, the fee does *not* include regular meals, travel or accommodation.

We support and empathize with those affected by the ongoing war in Ukraine, therefore we offer a **free participation** for affected individuals and teams affiliated with Ukrainian institutions.

→ **Pay attention**

Before contest
[Codeforces Round \(Div. 2\)](#)
4 days
[Register now »](#)
*has extra registration

→ **longnguyendt04**

★ Contribution: 0

- [Settings](#)
- [Blog](#)
- [Teams](#)
- [Submissions](#)
- [Talks](#)
- [Contests](#)

longnguyendt04

→ **Top rated**

#	User	Rating
1	tourist	3757
2	Benq	3738
3	jiangly	3705
3	Kevin114514	3705
5	orzdevinwang	3670
6	ksun48	3668
7	Radewoosh	3531
8	maroonrk	3443
9	dKqwq	3436

Figure 3.2: Codeforces Homepage

across multiple difficulty levels.

- **Online Contests:** Regular timed competitions, including rated contests, that simulate real-world competitive programming scenarios.
- **Integrated Code Editor:** Supports multiple programming languages with submission, testing, and evaluation against problem test cases.
- **Community Discussion:** Active forums for problem discussions, solution sharing, and peer feedback.
- **User Ratings and Rankings:** Automatic rating system reflecting performance in contests and overall ranking among users globally.
- **Problem Tags and Difficulty Filters:** Tools to browse and search problems efficiently based on difficulty, topic, and tags.

Codeforces excels in providing a practical, competition-oriented environment that strengthens real-world programming and algorithmic skills. Its focus on timed contests and rigorous problem solving makes it an excellent platform for users aiming to improve performance in competitive programming. However, Codeforces does not currently provide dedicated tools for technical interview preparation, such as virtual assistants, hints, or structured interview question sets.

3.1.3 Websites comparison

Obviously, each platform has its own strengths: LeetCode offers an extensive problem library and dedicated interview preparation resources, while Codeforces emphasizes timed

Table 3.1: *Comparison of existing website with CodeRank*

Criteria	LeetCode	Codeforces	CodeRank
Problem Library	Yes	Yes	Yes
Online Coding Editor	Yes	Yes	Yes
Community	Active discussion forums	Active forums with contest discussions	Centralized real-time discussion forums
Contest System	Regular contests organized by platform	Timed contests with system-created schedule	Both official and user-created contests, customizable by users.
Interview Preparation	Yes	No	Yes
Ranking System	Points for solved problems, but not visible to all users	Rating system based on contest performance	Dynamic points for problem solving and public centralized leaderboard
Multilingual	Support English & Chinese	Support English & Russian	Support English & Vietnamese

contests, a rating system, and community-driven forums for competitive practice. However, CodeRank combines these advantages and adds key enhancements, including both official and user-created contests with customizable settings, a centralized public leaderboard, real-time discussion forums, and multilingual support (including Vietnamese). These features make CodeRank a comprehensive, user-centered platform tailored to the educational and practical needs of Vietnamese IT students.

3.2 Requirement analysis

3.2.1 Stakeholders

In any software system, understanding the stakeholders is essential for defining responsibilities, ensuring smooth operation, and aligning the platform with user needs. This project identifies three main groups of stakeholders, each playing a critical role in the development, management, and effective use of the platform. The following sections describe these groups in detail: Primary Users, Moderators, and Superadmin.

1. Primary Users

Primary users consist of students, programming contestants, and job seekers who utilize the platform to enhance their coding abilities and evaluate their technical proficiency. These users engage with the system by practicing programming challenges, submitting solutions for automated evaluation, and participating in online contests that allow them to monitor their progress and compare performance with peers. Additionally, they may take

part in discussion forums to exchange ideas and seek guidance. The platform also provides access to an integrated virtual assistant designed to support interview preparation, enabling users to develop both competitive and career-readiness skills.

2. Moderators

Moderators serve as the quality control and management layer of the platform. Their responsibilities include reviewing problem statements, validating test cases, and monitoring ongoing contests to ensure fairness and a seamless experience for all participants. Moreover, moderators oversee the integrity of the system by detecting plagiarism, resolving disputes, and upholding community standards within discussion forums. Their involvement is essential for maintaining a trustworthy, transparent, and academically honest environment.

3. Superadmin

The Superadmin holds the highest level of authority within the system and is responsible for overarching platform management. In addition to performing all moderator functions, the Superadmin oversees system-wide operations, configures platform policies, and supervises both moderators and regular users. Their core duties include creating and managing moderator accounts, managing roles, addressing escalated issues, and monitoring system performance through analytics and operational metrics. This role ensures that the platform remains stable, secure, and aligned with organizational objectives.

3.2.2 Functional requirements

Primary Users

1. Authentication:

- The system shall allow users to create an account using email and password or third-party authentication providers (e.g., Google, GitHub).
- The system shall support secure login and logout functionality.
- The system shall provide password recovery and reset options.
- The system shall enforce role-based access control to differentiate between standard users, moderators, and superadmins.

2. Problem Solving and Submission:

- The system shall allow users to browse and search algorithmic problems by difficulty, topic, tags, and status (solved, attempted).
- The system shall provide an integrated code editor supporting multiple programming languages (C++, Java, Python), enabling users to write, compile, and execute their code against provided example test cases.
- The system shall display immediate feedback on submitted solutions, indicating success/failure, execution time, memory usage, and specific error messages (e.g.,

Wrong Answer, Time Limit Exceeded, Runtime Error).

- The system shall allow users to view their past submissions and their results.

3. *Contest Participation:*

- The system shall enable users to create or participate in online programming contests. In case the users want to create an online contest, they can enforce the contest rules, such as time limits per problem and penalty for incorrect submissions.
- The system shall display a leaderboard after the contests finish.

4. *Community forum:*

- The system shall provide a shared discussion forum where users can exchange ideas, ask questions, and discuss solutions for all problems, similar to a general online community forum.
- The system shall allow users to upvote/downvote comments and report inappropriate content.

5. *Ranking System:*

- The system shall assign points to users upon successfully solving problems, with points varying according to the difficulty of each problem.
- The ranking system shall update user scores and global rankings every 15 minutes, reflecting users' latest achievements and progress.

6. *Interview Preparation (Virtual Assistant):*

- The system shall offer a virtual assistant feature for practicing interview questions (e.g., mock interviews with predefined scenarios).
- The system shall provide hints and feedback on interview responses.

Moderators

1. *Problems Management:*

- The system shall allow moderators to create, edit, and publish new algorithmic problems.
- The system shall enable moderators to define problem statements, input/output formats, constraints, and sample test cases.
- The system shall allow moderators to upload and manage hidden test cases for problem evaluation.

2. *Contest Management:*

- The system shall allow moderators to receive and review requests from users for creating new contests. Moderators shall have the ability to either approve or reject these contest creation demands, providing a reason for rejection if applicable. Upon approval, the system shall notify the requesting user and enable the moderator to proceed with configuring and publishing the contest.

3. *Community Moderation:*

- The system shall allow moderators to review and manage content in discussion fo-

rums, including deleting inappropriate posts or comments.

- The system shall enable moderators to warn or ban users who violate community guidelines.

Superadmin

The Superadmin shall possess all the functional capabilities and permissions granted to a moderator.

1. *User and Moderator Account Management:*

- The system shall allow the Superadmin to create, edit, suspend, and delete user and moderator accounts.
- The system shall enable moderators to warn or ban users who violate community guidelines.

3.2.3 Non-functional requirements

Performance: The system shall support at least 1,000 concurrent users while maintaining minimal latency during contests. Code submissions must be evaluated and feedback returned within 3 seconds for standard test cases, ensuring a responsive and smooth user experience.

Scalability: The system shall be designed to support horizontal scaling, allowing it to handle increasing numbers of users and higher workloads by adding additional servers or computational resources as needed.

Reliability: The system shall maintain high availability, particularly during contests and peak usage periods. Mechanisms for fault tolerance and recovery shall be implemented to minimize downtime and ensure continuous operation.

Security: The system shall safeguard user accounts, code submissions, and any sensitive data (including payment information) through industry-standard security measures, such as encryption, secure authentication, and role-based access control.

Usability: The system shall provide an intuitive and user-friendly interface suitable for both beginners and advanced users. Users should be able to quickly locate problems by difficulty or topic, join contests with minimal steps, and utilize the integrated code editor without additional setup.

Maintainability: The system shall be designed for ease of maintenance, allowing developers to update features, fix bugs, and enhance functionality with minimal effort and disruption.

Internationalization & Localization: The system shall support multiple languages and provide seamless switching between them, ensuring accessibility and usability for a diverse, global user base.

3.3 Use Cases

3.3.1 General Use Case

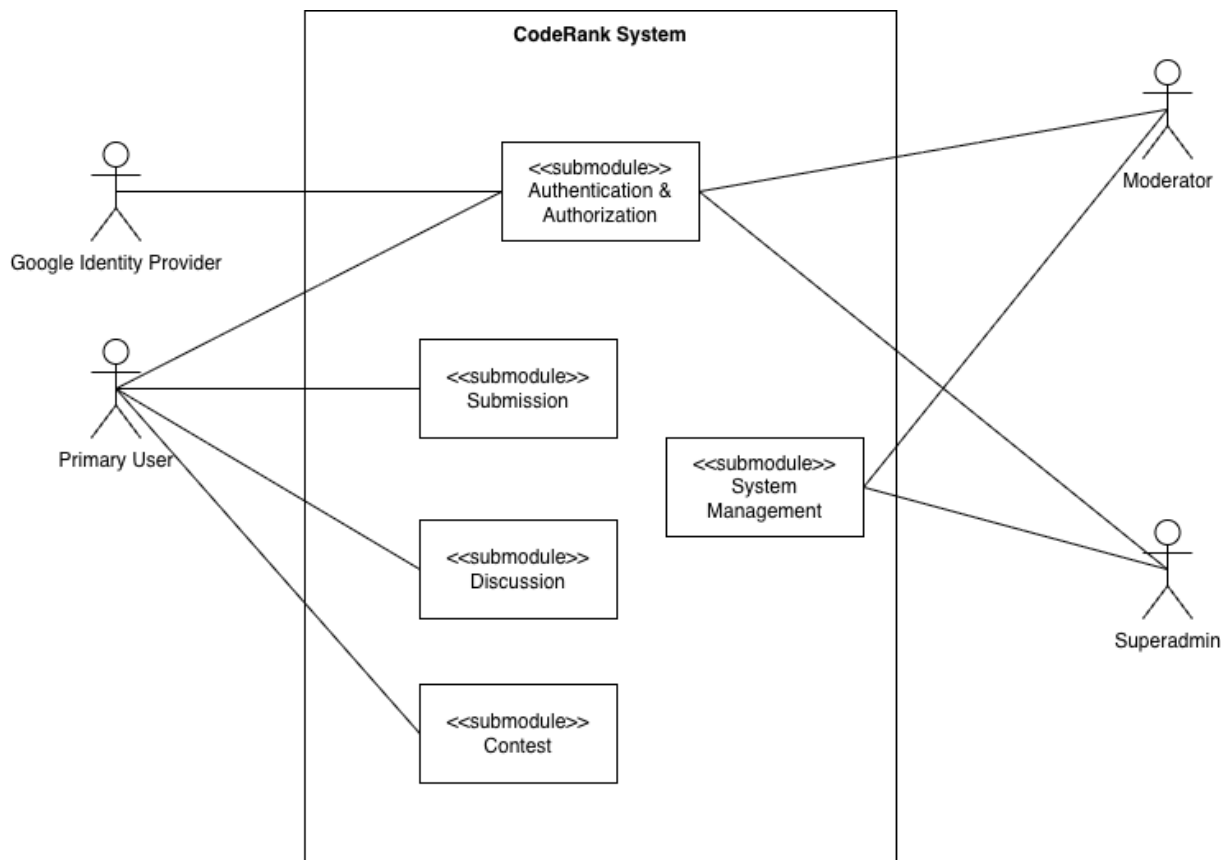


Figure 3.3: General Use Case

Our system contains 5 main submodules:

- **Authentication & Authorization:** Responsible for managing secure login and enforces Role-Based Access Control for 3 types of user.
- **Discussion:** Facilitates community interaction, allowing primary users to share solutions and ask questions related to problems.
- **System Management:** The administrative interface used by Moderator and Superadmin to manage the problem creation, test cases, contests management and global user accounts. Contest: Enables timed, high-stakes programming events with real-time ranking and automated leaderboards.
- **Submission:** This core module handles the presentation of the coding challenge and the

automatic evaluation of the user's submitted solution.

3.3.2 Use-case diagrams for each submodule

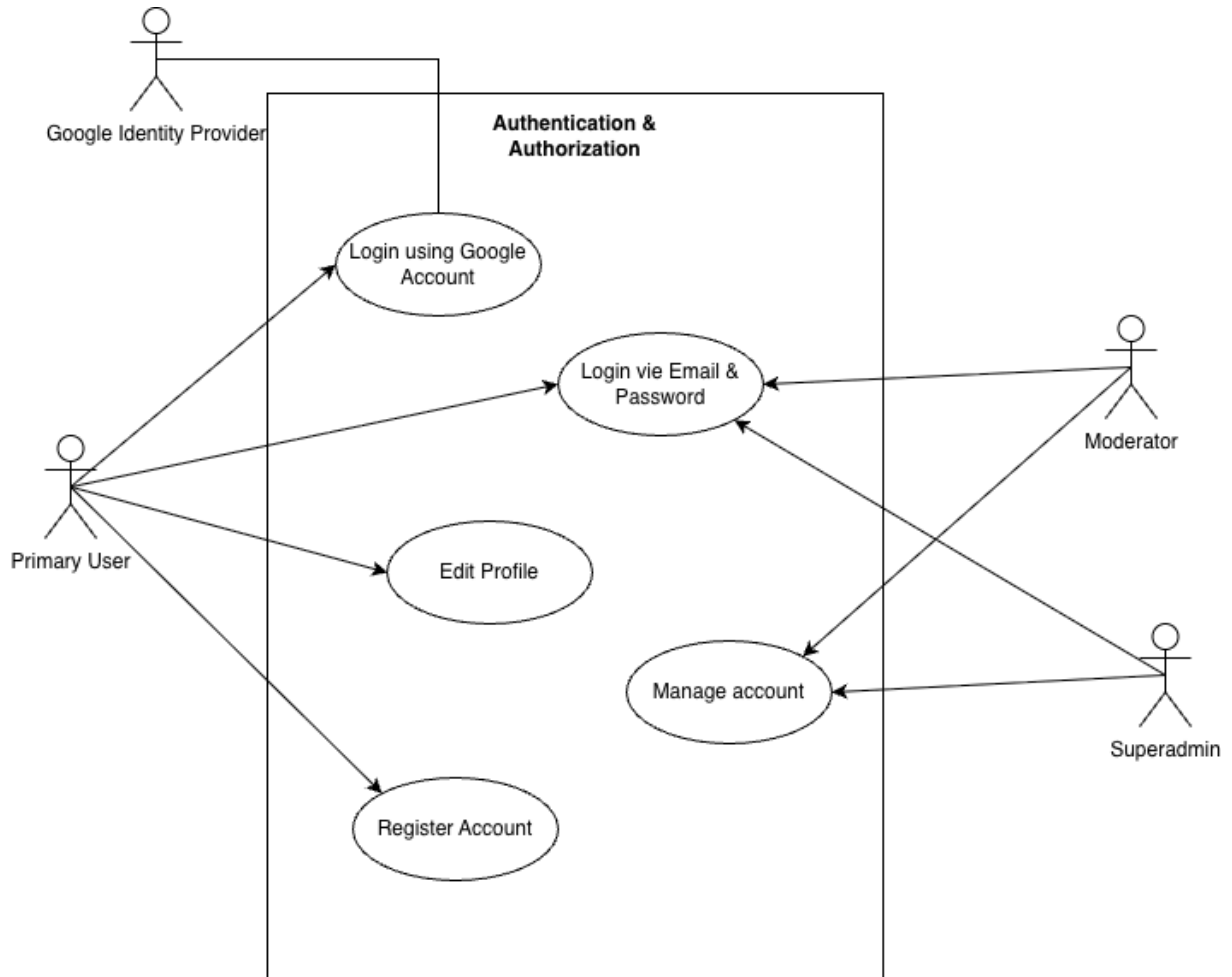


Figure 3.4: *Authentication & Authorization Use Case*

This submodule serves as the primary security gate for the entire system. Its core responsibility is to handle all user authentication (logging in) and authorization (managing access rights). This is crucial for protecting users' personal information, ensuring data privacy, and securing sensitive actions within the platform. By strictly controlling access, the module prevents unauthorized entry and maintains system integrity. For user convenience, we offer diverse login methods:

- Primary Users can log in quickly using their Google Account or use the traditional **Email & Password** method.
- However, for privileged administrative roles—the Moderator and the Superadmin—access is strictly limited to the Login via **Email & Password** method to enforce heightened security and traceability for system management functions.

All users retain standard account management capabilities, including registering a new ac-

count and changing their password.

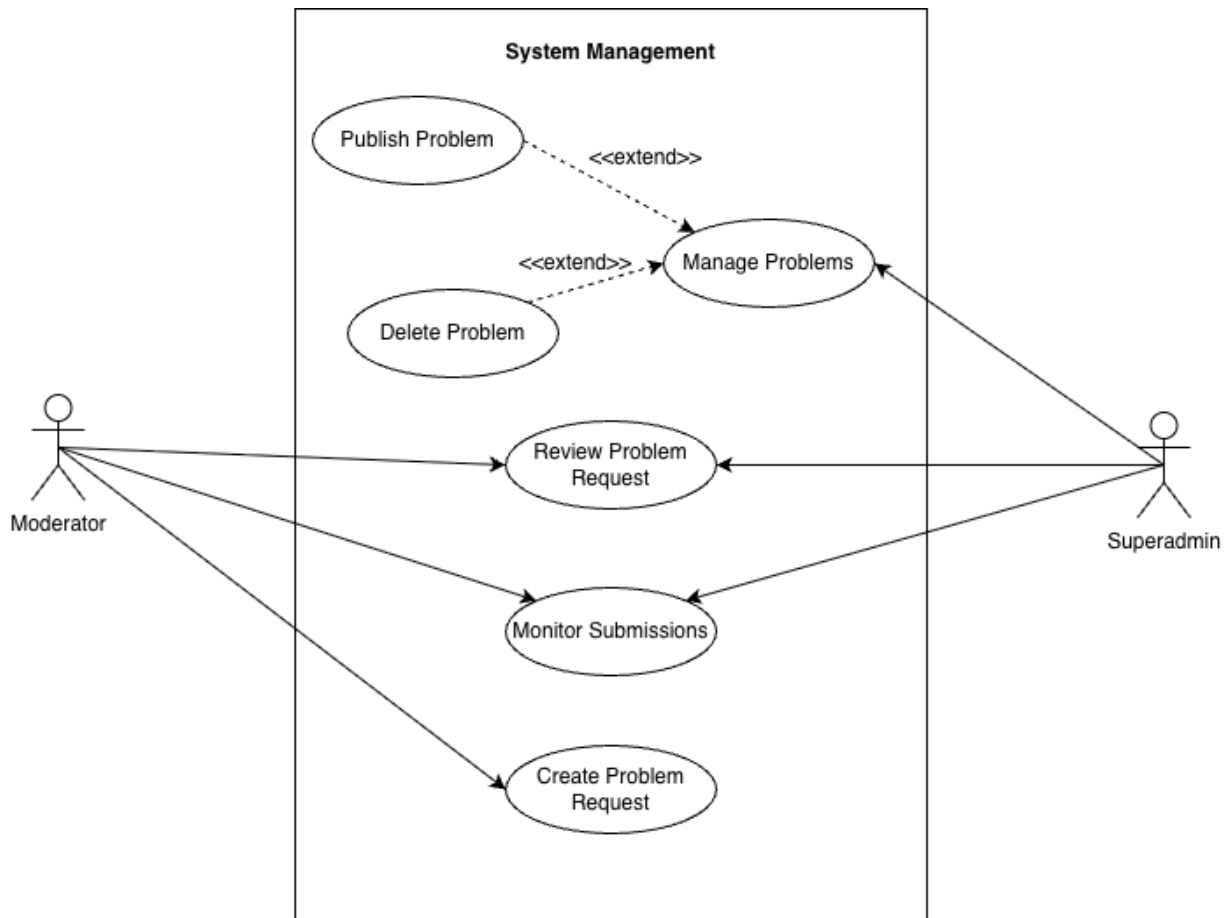


Figure 3.5: *System Management Use Case*

The **System Management submodule** is the administrative dashboard used exclusively by the **Moderator** and **Superadmin** roles to control the content and operation of the platform.

- **Problem Management:** Only Superadmin is enabled to manage problems directly (create, modify, delete) without reviewing steps
- **Content Workflow:** The Moderator initiates a Create Problem Request, and both the Moderator and Superadmin review problem requests before they go live.
- **Quality Control:** Both roles Monitor Submissions to track user activity, diagnose potential grading issues, and ensure fair play.

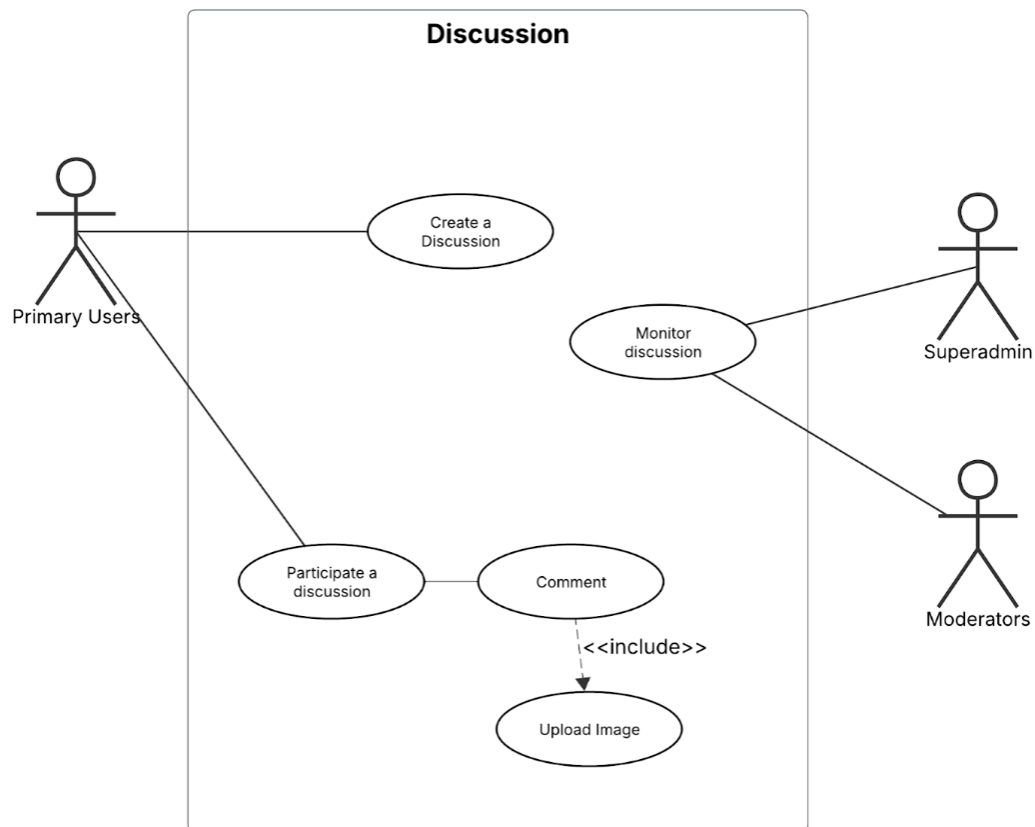


Figure 3.6: *Discussion Use Case*

The **Discussion submodule** helps make CodeRank a friendly and interactive community where users can learn from each other. With the Create Discussion feature, users can start new discussion threads about coding problems, programming ideas, or any tech-related topic. After a thread is created, others can join in by commenting on that thread. Comments can include rich content, such as images added, making explanations clearer and easier to follow. To ensure that discussions remain constructive and aligned with community guidelines, the submodule integrates essential administrative controls. Both the Superadmin and Moderators utilize the Monitor Discussion function to oversee user activity, identify and address inappropriate content, and uphold the platform's community standards.

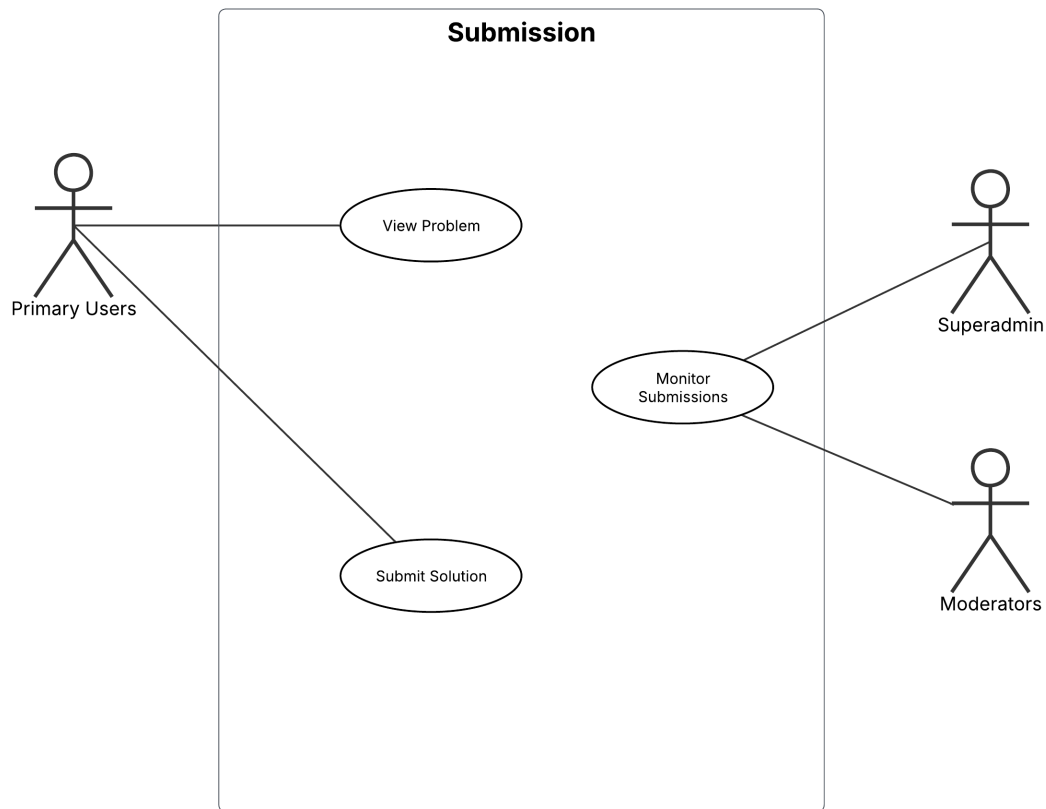


Figure 3.7: *Submission Use Case*

The **Submission** submodule is the core execution engine and interaction point for all coding activities within the CodeRank system. Its primary function is to facilitate the practice and assessment workflow for the Primary User. Users begin by accessing the **View Problem** function, which presents the algorithmic challenge and requirements. The fundamental action in this module is the **Submit Solution** function, which securely receives the user's code, initiates the automatic grading process in a sandboxed environment, and returns the execution verdict. For oversight and quality assurance, both the Superadmin and Moderators are granted the ability to Monitor Submissions. This administrative function is vital for diagnosing issues with test cases, tracking potential cases of plagiarism, and ensuring the overall stability and fairness of the automated grading system.

3.3.3 Use-case description tables

ID and Name	UC-1. Problems Management
Date Created	20th September, 2025
Primary Actor	Primary User, Superadmin, Moderator
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how Moderators and Superadmins manage algorithmic problems on the platform. The process includes creating, editing, approving, and publishing problems. It also covers defining problem statements, input/output formats, constraints, sample test cases, and hidden test cases.
Trigger	A Moderator or Superadmin decides to create, edit, approve, or publish a problem.
Preconditions	- The user is logged in as a Moderator or Superadmin. - The problem management interface is accessible.
Postconditions	- A new problem is created and saved as draft. - A draft problem is updated or submitted for approval. - A published problem may be updated after Superadmin approval. - A problem is published once the approval rule is satisfied.
Normal Flow	- Moderator/Superadmin accesses the problem management interface. - Moderator creates a new problem (saved as draft). - Moderator edits draft problem (auto-saved). - Moderator or Superadmin submits the problem for approval. - Moderator approves problem → System checks publishing rule. - If approvals ≥ 3 (Moderators) or 1 (Superadmin), system auto-publishes. - Superadmin can directly approve and publish a problem.
Alternative Flow	AF-1: Moderator edits a published problem → System saves changes as <i>pending_edit</i> → Requires Superadmin approval. AF-2: If approval threshold is not met, the problem stays pending until more approvals are collected.
Exceptions	E-1: Superadmin rejects a pending edit → System notifies Moderator. E-2: Any moderator rejects a pending edit → System notifies Moderator.

Table 3.2: UC-1: Problems Management

ID and Name	UC-2. Contest Management
Date Created	20th September, 2025
Primary Actor	Primary User, Superadmin, Moderator
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how users create and submit contests, allow administrators to review and approve them, and make contests available for participation once published.
Trigger	A Primary User submits a new contest request.
Preconditions	• The user is authenticated. • Contest details (title, description, rules, problems) are provided. • The contest submission form is valid.
Postconditions	• Contest status is updated (pending, approved, or published). • Notifications are sent to users and admins. • Users can participate in contests once published.
Normal Flow	1. Primary User submits a contest request. 2. The System saves the contest as <i>Pending Review</i>. 3. Moderators review and approve the contest. 4. If at least 3 moderator approvals are received, the contest qualifies for publishing. 5. Alternatively, a Superadmin may approve and publish with a single approval. 6. The System updates the contest status to Published. 7. Notifications are sent to the contest creator and relevant moderators. 8. Primary Users may now participate in the contest.
Alternative Flow	• AF-1: Insufficient Approvals: If fewer than 3 moderator approvals are received, the contest remains in <i>Pending Review</i>. • AF-2: Superadmin Override: A Superadmin may publish directly, bypassing moderator approvals. • AF-3: Invalid Submission: If contest data is incomplete or invalid, the submission is rejected with an error message.
Exceptions	• E-1: Database error when saving or updating the contest request. • E-2: Approval action references a contest that does not exist. • E-3: Notification delivery fails (system logs error, but contest status remains updated).

Table 3.3: UC-2: Contest Management

ID and Name	UC-3. Primary User Account Registration
Date Created	20th September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User creates a new account through the Sign-Up interface. The process begins from the Sign-In page, where the user chooses to register instead. Upon successful registration, the system displays a confirmation toast and navigates the user back to the login page.
Trigger	A Primary User clicks the <i>Sign Up</i> button on the Sign-In page.
Preconditions	• The user is on the Sign-In page. • The user has not previously registered with the same email address. • The Sign-Up form is accessible.
Postconditions	• A new Primary User account is created in the system. • A success toast notification is displayed. • The system redirects the user to the Sign-In page.
Normal Flow	1. The Primary User is on the Sign-In page and clicks the <i>Sign Up</i> button. 2. The System displays the Sign-Up form. 3. The Primary User enters required information (e.g., email, password, full name). 4. The Primary User clicks the <i>Register</i> button. 5. The System validates the input fields. 6. If validation succeeds, the System creates a new user account. 7. The System shows a success toast message: "<i>Account created successfully!</i>" 8. The System redirects the user back to the Sign-In page.
Alternative Flow	• AF-1: Invalid Input — If required fields are missing or invalid, the System highlights the errors and prevents submission. • AF-2: Email Already Exists — If the email is already registered, the System displays an error toast and remains on the Sign-Up page.
Exceptions	• E-1: System error occurs during account creation → System shows an error toast, and the user remains on the Sign-Up page. • E-2: Network timeout during submission → System displays a retry message.

Table 3.4: Use Case UC-3: Primary User Account Registration

ID and Name	UC-4. Problem Browsing and Searching
Date Created	21st September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User browses the list of available algorithmic problems. Users can view problems in a paginated or scrollable list, search by keywords, filter by tags or difficulty, and sort by attributes such as popularity, difficulty, or newest updates.
Trigger	The Primary User navigates to the <i>Problem List</i> page.
Preconditions	• The user is authenticated as a Primary User. • The problem database contains published problems.
Postconditions	• A filtered or sorted list of problems is displayed. • The user may select a problem to view full details.
Normal Flow	1. The Primary User opens the Problem List page. 2. The System displays a list of all published problems in default order (e.g., newest first). 3. The User enters text in the search bar. 4. The System filters and displays problems matching the search keyword. 5. The User applies optional filters (e.g., tags, difficulty, status). 6. The System updates the displayed list based on selected filters. 7. The User selects a sorting option (e.g., difficulty, popularity, newest). 8. The System sorts and refreshes the list. 9. The User may click on a problem to view its full details.
Alternative Flow	• AF-1: No Search Results — If no problem matches the search keyword, the System displays an empty list with a message. • AF-2: No Filter Results — If no problem matches selected filters, the System shows “No problems found”. • AF-3: Reset Filters — The User clicks <i>Reset Filters</i>, and the System returns to default problem list.
Exceptions	• E-1: Network or server error prevents loading the problem list → System shows an error message. • E-2: Search API request fails → System keeps previous results and displays a retry option.

Table 3.5: UC-4: Problem Browsing & Searching

ID and Name	UC-5. Solving a Problem
Date Created	21st September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User selects a problem, writes code using the built-in editor, runs sample test cases, and submits the solution for evaluation. The user receives feedback such as pass/fail results, execution time, and error messages, similar to platforms like LeetCode.
Trigger	The Primary User clicks on a problem from the Problem List page.
Preconditions	• The user is authenticated. • The selected problem is published and accessible. • The code editor and test case runner are available.
Postconditions	• The user's solution is evaluated by the system. • Submission results are recorded (pass/fail, time, memory). • The user may see their submission history for the problem.
Normal Flow	1. The Primary User opens a problem from the Problem List. 2. The System displays the problem description, examples, constraints, and editor. 3. The User writes or edits code in the editor. 4. The User clicks <i>Submit</i>. 5. The System runs the solution against all hidden test cases. 6. The System returns final evaluation results (Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error, etc.). 7. The System stores the submission result in the user's submission history.
Alternative Flow	• AF-1: Code Run Fails — The code fails during the sample run (syntax error, runtime error). System displays error messages without blocking further editing. • AF-2: Wrong Answer on Submission — Hidden test cases fail. System shows expected vs actual output. • AF-3: Time Limit Exceeded — System stops execution and informs the user. • AF-4: Memory Limit Exceeded — System terminates execution and displays a memory error.
Exceptions	• E-1: Code execution service unavailable → System displays an error and disables Run/Submit temporarily. • E-2: Submission timeout due to server overload → System requests the user to retry. • E-3: Problem deleted or unpublished during solving → User is redirected with a notification.

Table 3.6: Use Case UC-5: Solving a Problem

ID and Name	UC-6. Create a Discussion
Date Created	20th September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User initiates a new discussion thread, typically linked to a specific problem ID or a general topic, setting the initial content and title.
Trigger	The Primary User navigates to a problem page or the main forum and selects the "New Discussion" option.
Preconditions	• The user is authenticated. • The discussion creation interface is accessible.
Postconditions	• A new discussion thread is created with the status "Active." • The discussion is visible to other Primary Users.
Normal Flow	1. Primary User accesses the problem/forum page and initiates creation. 2. The user enters the discussion title and body content. 3. The user submits the discussion. 4. The System saves the new thread, assigns a unique ID, and displays it.
Alternative Flow	• AF-1: Content Validation Error: If the title or body content is empty, the System displays an error message and requires the User to correct the input before submission.
Exceptions	• E-1: Submission Failure: If the database connection fails, the System displays a technical error and saves the content as a temporary draft (if possible).

Table 3.7: UC-6: Create a Discussion

ID and Name	UC-7. Participate in a Discussion Thread
Date Created	21st September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User interacts within an existing discussion thread. The user may read comments, post replies, quote other users.
Trigger	The Primary User opens an existing discussion thread.
Preconditions	- The user is authenticated. - The selected discussion thread exists and is not locked.
Postconditions	- The user's reply, comment, or reaction is saved and added to the thread. - The thread updates to display the new interaction.
Normal Flow	- The Primary User opens a discussion thread. - The System displays all posts, replies, and thread structure. - The User types a reply in the text editor. - The User submits the reply. - The System validates the reply content. - The System saves the reply and assigns it a timestamp and unique ID. - The System refreshes the thread to show the new reply. - Other users can now view the new interaction.
Alternative Flow	AF-1: Reply to a Specific Comment — The user clicks <i>Reply</i> on a comment; the System nests the reply under that comment. AF-2: Quote a Comment — The user selects <i>Quote</i>; the System inserts the original text into the editor. AF-3: No Content Entered — If the reply box is empty, the System shows an error and prevents posting.
Exceptions	E-1: Thread Locked — A moderator locks the thread; the user cannot post replies. E-2: Submission Failure — System cannot save reply due to server/database error. E-3: Reply Deleted During Interaction — If a referenced comment is removed, the System notifies the user.

Table 3.8: UC-7: Participate in a Discussion Thread

ID and Name	UC-8. Edit Profile
Date Created	21st September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User updates their personal profile information, such as name, profile picture, or other optional fields. Changes take effect immediately after validation and confirmation.
Trigger	The Primary User navigates to the <i>Profile</i> page and selects <i>Edit Profile</i> .
Preconditions	- The user is authenticated. - The profile editing interface is accessible.
Postconditions	- Updated profile information is saved to the system. - The changes are displayed immediately on the user's profile.
Normal Flow	- The Primary User opens the Profile page. - The User clicks <i>Edit Profile</i>. - The System displays editable fields (e.g., name, bio, avatar). - The User updates one or more fields. - The User clicks <i>Save Changes</i>. - The System validates the updated information. - The System saves the changes and shows a success toast message. - The System reloads the Profile page with updated values.
Alternative Flow	AF-1: No Changes Made — If the user makes no edits and clicks <i>Save</i>, the System shows “No changes detected.” AF-2: Cancel Edit — The user clicks <i>Cancel</i>; the System discards edits and returns to the Profile page. AF-3: Invalid Image Upload — If an unsupported file is uploaded as an avatar, the System rejects it and requests a valid format.
Exceptions	E-1: Save Failure — System error prevents saving updates; the System shows a technical error toast. E-2: Session Expired — User session ends mid-edit; System redirects to login page.

Table 3.9: UC-8: Edit Profile

ID and Name	UC-9. Participate in a Contest
Date Created	21st September, 2025
Primary Actor	Primary User
Secondary Actor	System (CodeRank Platform)
Description	This use case describes how a Primary User participates in an active programming contest. The user can view problems, solve them within the contest duration, submit solutions, and receive real-time contest scoring and ranking updates.
Trigger	The Primary User selects an active contest and clicks <i>Enter Contest</i> .
Preconditions	• The user is authenticated. • The contest is currently active (between start and end time). • The contest has at least one published problem. • The user is eligible to participate (not banned, not restricted).
Postconditions	• User submissions are recorded as part of the contest scoring. • The System updates the contest scoreboard with the user's results. • Contest ranking is finalized once the contest ends.
Normal Flow	1. The Primary User opens an active contest page and selects <i>Enter Contest</i>. 2. The System displays contest rules, timer, and the problem list. 3. The User selects a problem from the list. 4. The System displays the problem statement and code editor. 5. The User writes code and clicks <i>Run Code</i> to test it locally on sample test cases. 6. The User clicks <i>Submit</i>. 7. The System evaluates the submission using the contest's judging system. 8. The System updates the user's contest score and standing on the leaderboard. 9. The User returns to the problem list and continues solving other problems until the contest ends.

Alternative Flow	• AF-1: Late Entry — If the user enters after the official start time, the System adjusts the user's available time to match remaining contest time. • AF-2: Wrong Answer Penalty — Incorrect submissions may apply time penalties or reduce score, depending on contest rules. • AF-3: Contest Timeout — When the contest timer expires, the System prevents further submissions.
Exceptions	• E-1: Submission System Failure — The judging service fails and the System temporarily blocks submissions. • E-2: Network Disconnection — If the user loses internet access, the System warns that submissions cannot be processed. • E-3: Contest Paused or Cancelled — Admin action pauses or cancels the contest; the user is notified and interaction is restricted.

Table 3.10: *UC-9: Participate in a Contest*

ID and Name	UC-10. Chatbot Interview Conversation
Date Created	21st September, 2025
Primary Actor	Primary User
Secondary Actor	System (Chatbot Module)
Description	This use case describes how a Primary User practices interview skills through an interactive chatbot conversation. The chatbot begins by asking the user which topic they want to practice, then continues the interview based on the selected topic.
Trigger	The user opens the Chatbot Interview page and selects "Start Conversation".
Preconditions	- The user is authenticated. - The chatbot conversation interface is accessible.
Postconditions	- A full or partial interview conversation is completed. - Feedback or a summary may be displayed to the user.
Normal Flow	- The user starts the chatbot conversation. - The chatbot sends the first message: "Which topic would you like to practice?" - The user types a topic (e.g., algorithms, system design, behavioral). - The chatbot sends the first interview question for that topic. - The user replies with an answer. - The chatbot analyzes the answer and responds with: - feedback, or - a follow-up question, or - the next question - Steps 5–6 repeat until the conversation ends. - The chatbot concludes the session and may provide a summary.
Alternative Flow	AF-1: User Changes Topic — The user requests a new topic; the chatbot restarts with a new topic question. AF-2: User Asks for Hint — The chatbot provides a hint instead of moving to the next question. AF-3: User Skips Question — The chatbot loads the next question for the same topic. AF-4: User Ends Conversation — The chatbot ends the session and displays partial feedback.
Exceptions	E-1: No User Response — If the user is inactive, the chatbot prompts again or ends the session after a timeout. E-2: Invalid Topic — The chatbot asks the user to choose or type a valid topic.

Table 3.11: UC-10: Chatbot Interview Conversation

3.4 Architecture Diagram

3.4.1 Architectural Decision Rationale

Architectural diagrams provide a structured view of how a software system is organized and how its components interact. In modern software development, two primary architectural styles are commonly considered: **monolithic architecture** and **microservice architecture**.

A **monolithic architecture** is a traditional model of a software program, which is built as a unified unit that is self-contained and independent from other applications. The word “monolith” is often attributed to something large and glacial, which isn’t far from the truth of a monolith architecture for software design. [1]

A **microservices architecture**, also simply known as microservices, is an architectural method that relies on a series of independently deployable services. These services have their own business logic and database with a specific goal. Updating, testing, deployment, and scaling occur within each service. Microservices decouple major business, domain-specific concerns into separate, independent code bases. [1]

As can be seen, the choice between these architectural approaches depends on the project’s specific requirements, team size, and long-term goals. Smaller systems or teams often benefit from the straightforwardness of monolithic architecture, while larger, more complex systems may require the flexibility and scalability of distributed services.

After evaluating our project goals and team size, we decided to implement a service-based modular monolith as the core architectural foundation for CodeRank. The key reasons for selecting this approach include:

- **Team Size and Feasibility** With only two members, adopting a full microservice architecture would introduce significant overhead in DevOps, deployment, and service management. In contrast, a modular monolith allows us to focus on implementing core features without the burden of distributed complexity.
- **Easier Development and Maintenance** By separating the system into modules—such as the Discussion Module, Submission Module, Auth Module, and Contest Module—we can maintain clean boundaries within a single codebase. This structure enables faster development cycles and reduces merging conflicts.
- **Future Scalability** While CodeRank currently operates as a monolithic application, its modules are designed following **Domain-Driven Design (DDD)** principles. These boundaries can later be extracted into microservices if the system needs to scale. This ensures long-term flexibility without requiring microservices prematurely.

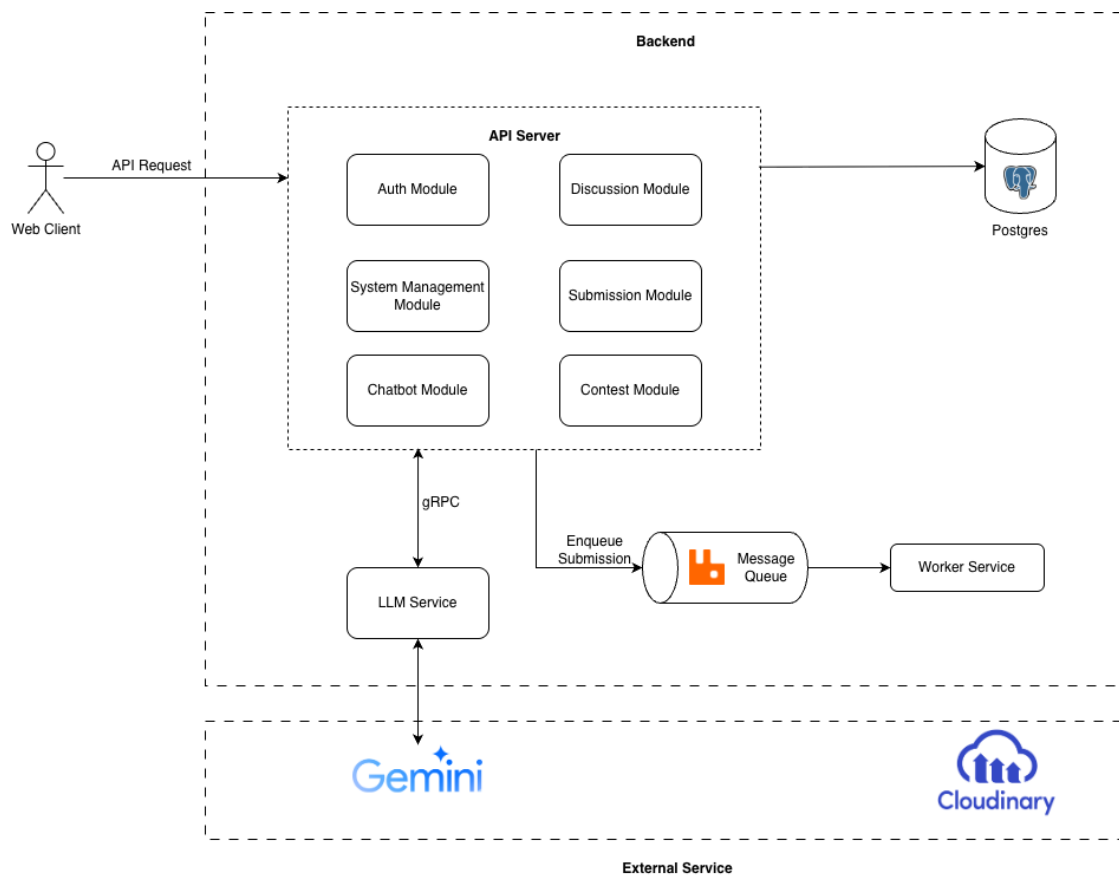


Figure 3.8: *Architecture Diagram*

3.4.2 System Components

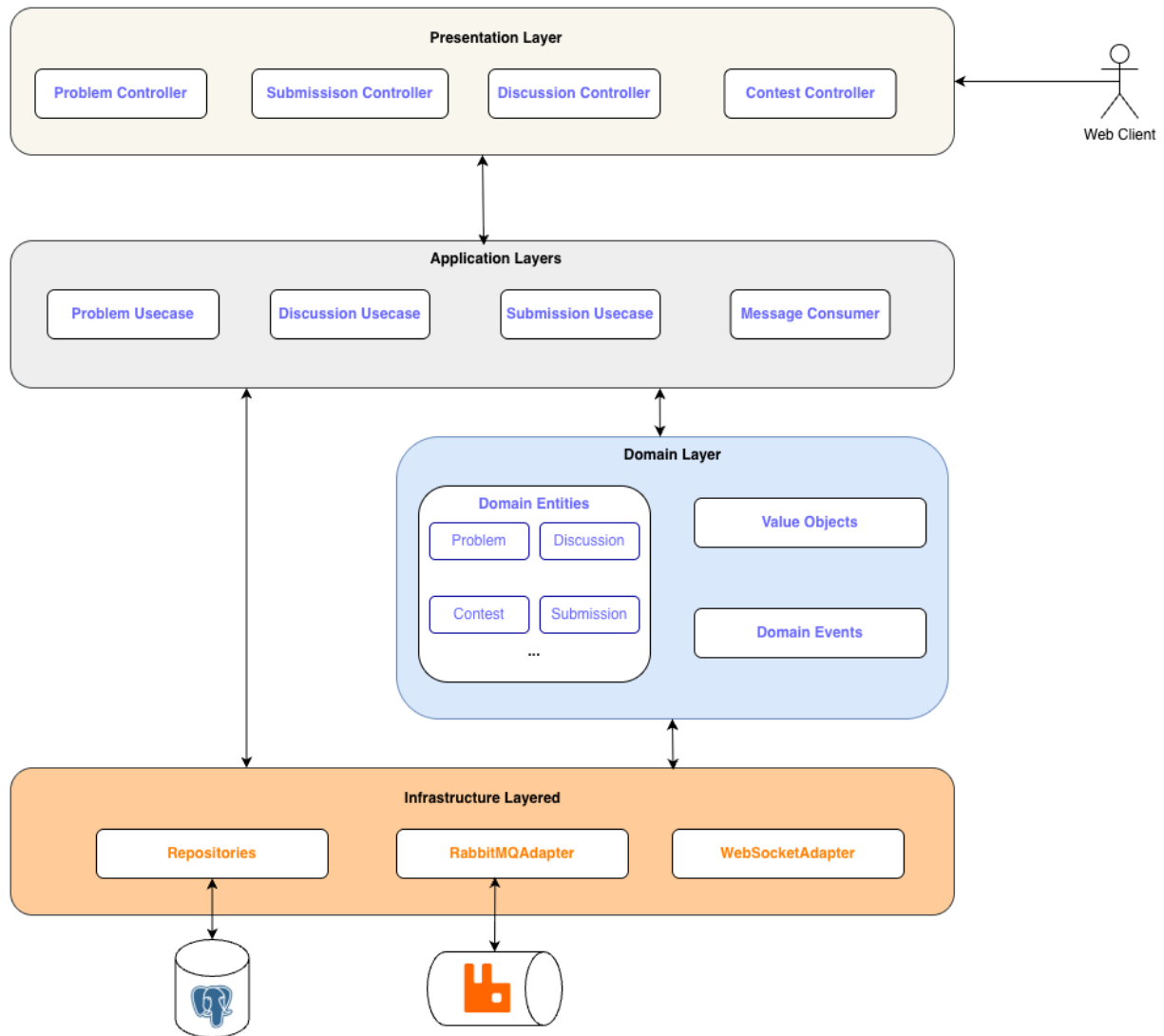


Figure 3.9: Layered Architecture

The system architecture of CodeRank is organized into four primary layers, each responsible for a specific set of functions. This layered approach ensures clean separation of concerns, maintainability, and a clear mapping between business rules and system implementation.

1. Presentation Layer

The **Presentation Layer** acts as the entry point for all external interactions with the system. It exposes various controllers such as the *Problem Controller*, *Submission Controller*, *Discussion Controller*, and *Contest Controller*, which handle incoming HTTP

requests from the web client. These controllers validate user input, map requests to the appropriate application use cases, and return responses in a structured format. This layer focuses purely on communication and does not contain business logic, ensuring the interface remains thin and easily adaptable.

2. Application Layer

The **Application Layer** contains the system's core orchestrators, known as Use Cases. Examples include the Problem Usecase, Discussion Usecase, Submission Usecase, and a dedicated Message Consumer for asynchronous tasks.

This layer coordinates operations between the Presentation Layer and the Domain Layer, ensuring that business rules are executed in the correct sequence. It does not implement business rules itself; instead, it acts as a mediator, invoking domain logic and triggering events. This design improves code clarity and supports future expansion.

3. Domain Layer

The **Domain Layer** represents the heart of the application, where all business logic and system rules are defined. It contains:

- Domain Entities such as Problem, Discussion, Contest, and Submission.
- Value Objects that encapsulate immutable attributes or concepts. Domain Events used to capture and react to significant changes within the system.
- Domain Events used to capture and react to significant changes within the system.

4. Infrastructure Layer

The **Infrastructure Layer** provides the technical capabilities required to support the domain and application layers. It includes:

- **Repositories**, which manage communication with the PostgreSQL database.
- **RabbitMQ Adapters**, responsible for publishing and consuming messages for asynchronous processing. Domain Events used to capture and react to significant changes within the system.
- **WebSocket Adapters**, enabling real-time communication such as live updates for submissions or discussion activities.

This layer abstracts external systems and framework-specific operations, ensuring that the upper layers remain decoupled from technical implementation details.

3.5 Code Runner

3.5.1 Overview

Code Runner Component is a core component of Submission Worker Service, responsible for executing user-submitted programs against predefined test cases. It serves as a bridge between the user's solution and the problem's input-output data, ensuring fair and consistent evaluation. For each programming language supported by the platform (C++, Java, Python),

a specific implementation of the **Code Runner** will be developed to handle language-specific compilation, execution, and output parsing. This ensures that user code is run correctly and consistently, regardless of the language chosen.

3.5.2 Class Diagram

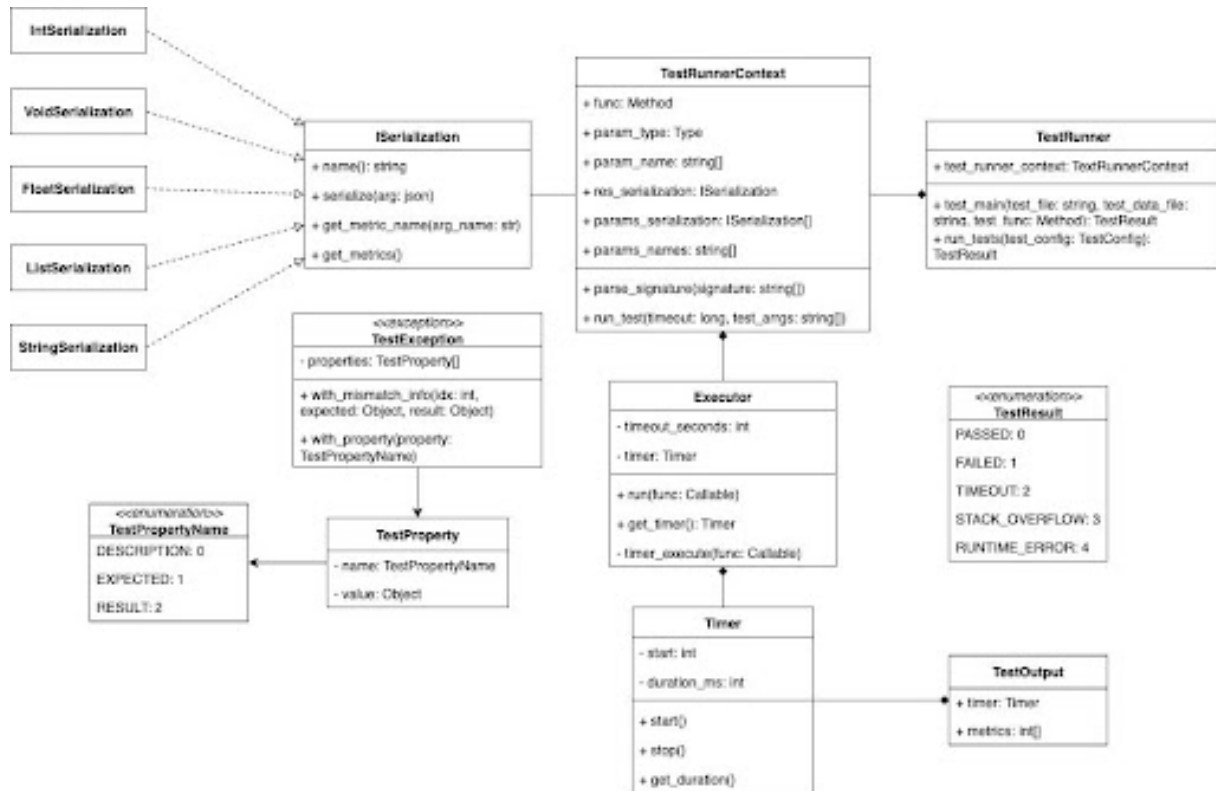


Figure 3.10: Class Diagram for Code Runner

- **ISerialization & Implementations:** ISerialization defines a common interface for converting data between raw test inputs/outputs and program variables. Its implementations — IntSerialization, FloatSerialization, StringSerialization, ListSerialization, and VoidSerialization — handle serialization for different data types, ensuring test cases are interpreted correctly regardless of format.
- **TestRunnerContext:** TestRunnerContext stores the configuration and metadata needed to execute a user's function, including parameter types, serializers, and the function reference itself. It prepares the environment for test execution and delegates the actual running to the Executor.
- **Executor:** Executor manages controlled execution of user-submitted functions. It enforces time limits, measures execution duration using Timer, and handles potential runtime issues like timeouts or errors during function calls.
- **Timer:** Timer measures how long each test case takes to execute. It provides simple start, stop, and duration tracking methods to record performance metrics for user solutions.

- **TestOutput:** TestOutput captures the results of a single test execution, storing both the runtime duration and any performance metrics. It is used to report how efficiently and accurately a user's code performed.
- **TestException:** TestException represents errors or mismatches that occur during test execution. It stores details such as expected versus actual results and additional context through TestProperty objects.
- **TestProperty & TestPropertyName:** TestProperty stores metadata about a test, such as a description or result comparison, identified by a TestPropertyName enum. These properties provide structured details for debugging failed test cases.
- **TestRunner:** TestRunner is the main controller responsible for running user code against predefined test cases. It coordinates between the TestRunnerContext, Executor, and ISerialization system to execute tests and produce final TestResult outcomes.
- **Solution:** Solution is the submission for the problem from the user.

3.5.3 Testcase Format

Each problem in the Online Judging System is accompanied by a test case file, named *problem_slug.csv*. This file defines a structured list of input–output test pairs used by the Test Runner of Worker Service to evaluate user submissions automatically. Moderators or superadmin upload this file when creating or updating a problem. The judging system parses the CSV, runs user code against each test case, and compares the output with the expected result.

To ensure accurate and consistent evaluation of user submissions, each input and output value in the *problem_slug.csv* file is associated with a data type. The type system defines how each field should be parsed, validated, and compared during the judging process. This mechanism helps the Test Framework interpret user program outputs correctly, whether they are numbers, strings, booleans, or lists. The following table describes the type. Here is the content for the "Problem Storages" section, including the table about type format:

Type Token	Description
int	Integer numbers.
float	Floating-point numbers.
boolean	Boolean data.
string	Textual data.
void	Represents the absence of a value (e.g., for functions that return nothing).
list(type)	A collection of elements of a specified type (e.g., list(int)).

Table 3.12: Data Types for Testcase Fields

Each problem's test case file will contain both type definitions for input, expected output and test data. The first line of the file specifies the data types for all inputs and outputs, helping

the Test Runner correctly parse and validate values before running tests.

After executing all test cases for a user's submission, the Test Runner generates a structured JSON result file summarizing the outcomes of each test. This file serves as the standardized output for the judging system and can be used by other services (such as the Result Service or API Gateway) to record, display, or analyze user performance. The JSON format ensures that results are machine-readable, language-independent, and easy to integrate into the overall online judging pipeline. The exported JSON file contains both metadata (e.g., problem name, language, execution status) and detailed per-test results (e.g., input, output, expected result, duration, and status).

Chapter 4

Future Plan and Work Evaluation

In Chapter 4, the authors will elucidate forthcoming plans and intentions for the next phase as well as evaluate the quality of the team's efforts.

Bibliography

- [1] Atlassian. Microservices vs monolith. <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>, n.d. Accessed: 2025-12-02.
- [2] AWS Firecracker Team. Firecracker: Lightweight virtualization for serverless computing. <https://firecracker-microvm.github.io/>, 2025. Accessed: 2025-12-02.
- [3] Proofpoint, Inc. What is a sandbox? <https://www.proofpoint.com/us/threat-reference/sandbox>, n.d. Accessed: 2025-12-02.